

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <title>Arrow Rendering Test</title>
  <style>
    body {
      font-family: sans-serif;
      display: grid;
      place-items: center;
      min-height: 90vh;
      background-color: #f0f0f0;
    }
    #mock-board {
      width: 400px;
      height: 400px;
      /* This mimics the alternating
square colors of a chessboard */
```

```
background-image:
    linear-gradient(45deg, #b58863
25%, transparent 25%),
    linear-gradient(-45deg, #b58863
25%, transparent 25%),
    linear-gradient(45deg, transparent
75%, #b58863 75%),
    linear-gradient(-45deg,
transparent 75%, #b58863 75%);
background-color: #f0d9b5;
background-size: 100px 100px; /* 8
squares on 400px = 50px per square. 2x
for pattern */
position: relative;
border: 2px solid #333;
}
.fancyground-arrows {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
```

```
    height: 100%;  
    overflow: visible;  
}  
button {  
    margin-top: 20px;  
    padding: 10px 20px;  
    font-size: 16px;  
    cursor: pointer;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Arrow Numbering Test</h1>
```

```
<div id="mock-board" class="orientation-  
white">
```

```
</div>
```

```
<div id="mock-shapes" style="display:  
none;">
```

```
<g cgHash="123,456,e2,e4,green">
  <line stroke-width="0.15"
opacity="0.7"></line>
</g>
<g cgHash="123,456,d7,d5,red">
  <line stroke-width="0.15"
opacity="0.7"></line>
</g>
<g cgHash="123,456,g1,f3,blue">
  <line stroke-width="0.15"
opacity="0.7"></line>
</g>
<g cgHash="123,456,b8,c6,yellow">
  <line stroke-width="0.15"
opacity="0.7"></line>
</g>
</div>

<button id="run-button">Draw Arrows</
button>
```

```
<script>
```

```
//
```

```
=====
```

```
=====
```

```
// == MOCK FUNCTIONS (To replace  
missing imports) ==
```

```
//
```

```
=====
```

```
=====
```

```
// Mocking type Num
```

```
type Num = 0 | 1 | 2 | 3;
```

```
// Mocking getColorNum from '../../  
defaults'
```

```
function getColorNum(colorName) {  
  if (colorName === 'green') return 1;  
  if (colorName === 'red') return 2;  
  if (colorName === 'blue') return 3;  
  if (colorName === 'yellow') return 0;  
  return 1; // Default
```

```
}
```

```
    // Mocking getColor and getOpacity  
    from '../..state'  
    function getColor(type, colorNum) {  
        const colors = ['#CCCC00',  
        '#00CC00', '#CC0000', '#0000CC']; //  
        Yellow, Green, Red, Blue  
        return colors[colorNum] || '#00CC00';  
    }
```

```
    function getOpacity(type) {  
        return '0.9'; // Made it less  
transparent for the test  
    }
```

```
    // Mocking keyToXY from '../common'  
    // This is the most important mock. It  
converts "e2" to pixel (x,y) coordinates.  
    // The SVG viewBox is "-0.5 -0.5 8 8",  
so we map 'a1' to {x:0, y:7} and 'h8' to {x:7,
```

```
y:0}
```

```
function keyToXY(key, flip) {  
    const file = key.charCodeAt(0) - 97;
```

```
// a=0, b=1, ... h=7
```

```
    const rank = parseInt(key[1]) - 1; //
```

```
1=0, 2=1, ... 8=7
```

```
    // SVG Y-coordinate is inverted (0 is  
top, 7 is bottom)
```

```
    const y_svg = 7 - rank;
```

```
    if (flip) {
```

```
        return { x: 7 - file, y: 7 - y_svg }; //
```

```
Flip both
```

```
    }
```

```
    // Add 0.5 to center the arrow in the  
square
```

```
    return { x: file, y: y_svg };
```

```
}
```

```
// Mocking createDefs, createMask,
```

```
getCap from './defs/render'
function createDefs(prefix) {
  const defs =
document.createElementNS('http://
www.w3.org/2000/svg', 'defs');

  // Create a simple arrowhead for
each color
  [0, 1, 2, 3].forEach(num => {
    const marker =
document.createElementNS('http://
www.w3.org/2000/svg', 'marker');
    marker.setAttribute('id', `${prefix}
arrowhead-${num}`);
    marker.setAttribute('orient', 'auto');

marker.setAttribute('markerWidth', '3');

marker.setAttribute('markerHeight', '4');
    marker.setAttribute('refX', '2.05');
    marker.setAttribute('refY', '2.01');
```



```
        const path =  
document.createElementNS('http://  
www.w3.org/2000/svg', 'path');  
        path.setAttribute('d', 'M0,0 V4 L3,2  
Z');  
        path.setAttribute('fill',  
getColor('arrow', num));  
  
        marker.appendChild(path);  
        defs.appendChild(marker);  
    });  
    return defs;  
}
```

```
function createMask(uci, flip, prefix,  
board, svg) {  
    return null; // Do nothing for this test  
}
```

```
function getCap(prefix, colorNum) {
```

```
        return `url(#${prefix}arrowhead-${colorNum})`;
    }

```

```
//

```

```
=====

```

```
=====

```

```
// == YOUR CODE (with modifications)

```

```
==

```

```
//

```

```
=====

```

```
=====

```

```
function updateArrows(shapes, board, prefix) {

```

```
    const svg =
board.querySelector('svg') ||
createNewSvg(board);

```

```
    svg.innerHTML = "";

```

```
svg.insertAdjacentElement('afterbegin',

```

```
createDefs(prefix));
    const arrows =
shapes.querySelectorAll('g[cgHash]');
    const flip = !!
board.closest('.orientation-black');

    for (let i = 0; i < arrows.length; i++) {
        const arrowElement =
arrows[i].firstElementChild;
        if (!arrowElement ||
arrowElement.tagName.toUpperCase() !==
'LINE') continue;

        const cgHash =
arrows[i].getAttribute('cgHash');
        const cgHashParts =
cgHash.split(',').filter(val => val !== 'true');
        const uci = cgHashParts[2] +
cgHashParts[3];
        const colorNum =
getColorNum(cgHashParts[4]);
```

```
const coords = uciToCoords(uci,  
flip);
```

```
const maskId =  
createMask(uci.slice(0, 2), flip, prefix,  
board, svg);
```

```
// --- Changes from your request ---
```

```
// 1. Get the color *before*
```

creating the arrow

```
const color = getColor('arrow',  
colorNum);
```

```
// 2. Pass the color into
```

createArrow

```
const arrow = createArrow(  
  shortenTip(coords),  
  svg,  
  getCap(prefix, colorNum),  
  colorNum,
```

```
arrowElement.getAttribute('opacity'),
    getArrowWidth(arrowElement),
    color // <-- Pass the color here
);

// 3. Call the new function to
create the number
// This logic creates pairs of
numbers (1,1, 2,2, 3,3...)
const moveNumber = Math.floor(i
/ 2) + 1;

createMoveNumber(moveNumber, coords,
color, svg);

    if (maskId) {
        arrow.setAttributeNS(null,
'mask', `url(#${maskId})`);
    }
}
}
```

// This function is not used in this test,
but included for completeness

```
function updateArrowColor(board,  
colorNum, newColor) {  
    const arrows =  
board.getElementsByTagNameNS('http://  
www.w3.org/2000/svg', 'line');  
    for (let i = 0; i < arrows.length; i++) {  
        const arrow = arrows[i];  
        if (arrow.dataset.colorNum === `${  
{colorNum}`) {  
            arrow.setAttributeNS(null,  
'stroke', newColor);  
        }  
    }  
    // ... (rest of function)  
}
```

// ... (other update functions) ...

```
function getArrowWidth(arrow) {  
    const widthStr =  
arrow.getAttribute('stroke-width');  
    if (!widthStr) return;  
    const width = Number(widthStr);  
    if (Number.isNaN(width)) return;  
    else if (width === 0.1328125) return  
0.15625;  
    else return width;  
}
```

```
function createNewSvg(board) {  
    const svg =  
document.createElementNS('http://  
www.w3.org/2000/svg', 'svg');  
    svg.classList.add('fancyground-  
arrows');  
    // Use '0 0 8 8' for simpler  
coordinates, mapping 'a1' to {0, 7}  
    svg.setAttribute('viewBox', '-0.5 -0.5
```

```
8 8');
```

```
    svg.style.opacity =  
getOpacity('arrow');
```

```
board.insertAdjacentElement('beforeend',  
svg);
```

```
    return svg;  
}
```

```
/**
```

```
 * Creates an SVG text element for the  
move number.
```

```
 * It draws a white "halo" behind the  
text to make it readable.
```

```
*/
```

```
function createMoveNumber(  
    moveNumber,  
    coords,  
    color,  
    svg  
) {
```



```
const text =  
document.createElementNS('http://  
www.w3.org/2000/svg', 'text');  
  
    // Position it at the center of the  
arrow's starting square (add 0.5)  
    text.setAttributeNS(null, 'x', `${  
coords.x1 + 0.5}`);  
    text.setAttributeNS(null, 'y', `${  
coords.y1 + 0.5}`);  
  
    // Style the text  
    text.setAttributeNS(null, 'font-size',  
'0.6');  
    text.setAttributeNS(null, 'font-  
weight', 'bold');  
    text.setAttributeNS(null, 'text-  
anchor', 'middle');  
    text.setAttributeNS(null, 'dominant-  
baseline', 'middle');  
    text.textContent = `${moveNumber}`;
```

// 1. Create the white "halo" (a clone of the text)

```
const halo = text.cloneNode(true);
halo.setAttributeNS(null, 'fill', '#fff');
halo.setAttributeNS(null, 'stroke',
'#fff');
halo.setAttributeNS(null, 'stroke-
width', '0.2');
```

```
svg.insertAdjacentElement('beforeend',
halo);
```

// 2. Create the colored text on top

```
text.setAttributeNS(null, 'fill', color);
```

```
svg.insertAdjacentElement('beforeend',
text);
}
```

// MODIFIED createArrow to accept

color

```
function createArrow(  
    coords,  
    svg,  
    cap,  
    colorNum,  
    originalOpacity,  
    width,  
    color // <-- New parameter  
) {
```

```
    const arrow =
```

```
    document.createElementNS('http://  
    www.w3.org/2000/svg','line');
```

```
        // const color = getColor('arrow',  
    colorNum); // <-- Removed
```

```
        // Add 0.5 to x/y to center in the  
    square
```

```
        arrow.setAttributeNS(null, 'x1', ` $  
    {coords.x1 + 0.5}`);
```

```
        arrow.setAttributeNS(null, 'y1', ` $
```

```
{coords.y1 + 0.5}`);  
    arrow.setAttributeNS(null, 'x2', `${  
{coords.x2 + 0.5}`);  
    arrow.setAttributeNS(null, 'y2', `${  
{coords.y2 + 0.5}`);  
    arrow.setAttributeNS(null, 'marker-  
end', cap);  
    arrow.setAttributeNS(null, 'stroke',  
color);  
    if (width) arrow.setAttributeNS(null,  
'stroke-width', `${width}`);  
    arrow.setAttributeNS(null, 'opacity',  
originalOpacity);  
    arrow.dataset.colorNum = `${  
{colorNum}`;  
  
svg.insertAdjacentElement('beforeend',  
arrow);  
    return arrow;  
}
```

```
function uciToCoords(uci, flip) {  
    const { 'x': x1, 'y': y1 } = keyToXY(uci,  
flip);  
    const { 'x': x2, 'y': y2 } =  
keyToXY(uci.slice(2), flip);  
    return { x1, y1, x2, y2 };  
}
```

```
function shortenTip({ x1, y1, x2, y2 }) {  
    // Treat coords as a vector from tail  
to tip  
    const dx = x2 - x1;  
    const dy = y2 - y1;  
    const magnitude = Math.sqrt(dx *  
dx + dy * dy);  
    // This trim amount is based on the  
8x8 viewBox  
    const trim = 160 / 512;  
    if (magnitude === 0) return { x1, y1,  
x2, y2 }; // Avoid division by zero  
    const scale = (magnitude - trim) /
```

```
magnitude;  
    return {  
        x1,  
        y1,  
        x2: x1 + dx * scale,  
        y2: y1 + dy * scale,  
    };  
}
```

```
//
```

```
=====
```

```
=====
```

```
// == SCRIPT EXECUTION ==
```

```
//
```

```
=====
```

```
=====
```

```
// Add a click listener to the button to  
run the code
```

```
document.getElementById('run-  
button').addEventListener('click', () => {
```

```
    const shapes =  
document.getElementById('mock-shapes');  
    const board =  
document.getElementById('mock-board');  
    const prefix = 'test-prefix';
```

```
    // This is the main function call  
    updateArrows(shapes, board,  
prefix);
```

```
        console.log("updateArrows()  
executed.");  
    });
```

```
    </script>  
</body>  
</html>
```